

Error Control and Data Compression

When data is transmitted from one device to another (e.g., from your phone to a server), two major problems can occur:

1. **Errors** – Noise may change bits during transmission.
2. **Large Data Size** – Data may be too big and slow to transmit.

To solve these problems, we use:

- **Error Control Techniques** → To ensure accuracy
- **Data Compression Techniques** → To reduce data size

Sometimes we only detect errors and request retransmission.

Example:

If a bank server detects a corrupted transaction request, it asks for it again.

Common error detection methods are:

- (i) Parity Checking
- (ii) Cyclic Redundancy Check (CRC)
- (iii) Checksum Error Detection

Parity Checking: Parity checking is an error detection technique in which an extra bit is appended to a data unit so that the total number of 1s in the transmitted data becomes either **even** or **odd**, depending on the type of parity used. The additional bit is called the **parity bit**. The parity bit helps the receiver verify whether the transmitted data has been altered during transmission. If the number of 1s received does not match the expected parity, the receiver concludes that an **error has occurred**.

Types of Parity Checking

Parity checking is generally divided into two main types: Even parity checking and Odd parity checking

1. **Even Parity:** In even parity, the parity bit is added so that the total number of 1s in the transmitted data (including the parity bit) becomes even.

Example 1:

Data to be transmitted: 1011001

Number of 1s = 4 (already even)

Therefore, the parity bit = **0**

Transmitted data becomes: 10110010

Example 2

Data to be transmitted: 1101011

Number of 1s = 5 (odd)

To make it even, parity bit = **1**

Transmitted data becomes: 11010111

2. **Odd Parity:** In **odd parity**, the parity bit is added so that the total number of 1s becomes **odd**.

Example 3

Data to be sent: 1010110

Number of 1s = 4 (even)

To make it odd, parity bit = **1**

Transmitted data: 10101101

How Parity Checking Works

Step 1: Sender Side

The sender counts the number of 1s in the data and adds a parity bit based on the chosen parity scheme (even or odd).

Step 2: Transmission

The data along with the parity bit is transmitted through the communication channel.

Step 3: Receiver Side

The receiver again counts the number of 1s in the received data.

Step 4: Error Detection

- If the parity condition is satisfied → data is assumed correct.
- If the parity condition is violated → an error is detected.

Limitations of Parity Checking

Although parity checking is simple and efficient, it has some limitations:

1. It can detect **only single-bit errors**.
2. It cannot detect errors when **two bits change simultaneously**.
3. It can **detect errors but cannot correct them**.

Because of these limitations, more advanced error detection techniques such as **Cyclic Redundancy Check (CRC)** and **Checksums** are often used in modern communication systems.

Applications of Parity Checking

Parity checking is commonly used in:

- Computer **memory systems (RAM error detection)**
- **Serial communication protocols**
- **Data storage devices**
- **Network transmission systems**

Advantages of Parity Checking

1. Simple to implement
2. Requires very little additional data (only one bit)
3. Fast error detection method
4. Low computational cost

Cyclic Redundancy Check (CRC)

Cyclic Redundancy Check (CRC) is a powerful error detection technique that uses **binary division** to detect errors in transmitted data. CRC works by treating the data as a binary number and dividing it by a predetermined **generator polynomial** using **modulo-2 division**. The remainder obtained from the division is called the **CRC code** or **Frame Check Sequence (FCS)**. This remainder is appended to the data before transmission. At the receiver side, the same division is performed. If the remainder is **zero**, the data is assumed to be correct.

Key Concepts in CRC

1. Dataword

The original data to be transmitted.

2. Generator Polynomial

A predetermined binary pattern used for division.

Example:

$$G = 1011$$

3. Codeword

The final transmitted data consisting of:

Dataword + CRC bits

Steps in CRC Error Detection

Step 1

Choose a generator polynomial.

Example:

$G = 1011$

Step 2

Append zeros to the data equal to **(length of generator – 1)**.

Step 3

Perform **modulo-2 division** (XOR operation)

Step 4

The remainder obtained becomes the **CRC bits**.

Step 5

Append the CRC bits to the original data and transmit.

CRC Calculation Example

Given Data = 1101011011

Generator = 10011

Length of generator = 5

Therefore append **4 zeros** to the data.

New data becomes:

1101011011 0000

Modulo-2 Division

11010110110000 ÷ 10011

Perform XOR operations instead of subtraction.

11010 ⊕ 10011 = 01001

Bring down next bit and continue division.

After completing the division, assume remainder obtained is: 1110

Transmitted Data (Codeword)

Original Data: 1101011011
CRC bits: 1110

Final transmitted data:

11010110111110

Receiver Side

The receiver divides the received data by the same generator polynomial.

If remainder = **0000**, then:

✓ No error detected

If remainder $\neq$ **0000**, then:

✗ Error detected

Advantages of CRC

1. Very reliable error detection technique
2. Detects burst errors effectively
3. Widely used in communication protocols

Applications of CRC

CRC is used in many communication systems including:

- Ethernet networks
- Hard disk drives
- USB communication
- Satellite communication
- Wireless networks

Checksum Error Detection

Checksum Error Detection is an error detection technique in which the data is divided into equal-sized segments and added together using binary addition. The sum obtained is then complemented and transmitted along with the data.

At the receiver side, the same addition is performed. If the final result is **all 1s**, the data is considered correct

Steps in Checksum Method

1. Divide the data into equal segments.
2. Add all segments using binary addition.
3. Take the **1's complement** of the result.

4. Send the data along with the checksum.
5. At the receiver side, add all segments including checksum.
6. If result = **all 1s**, then no error occurred.

Checksum Calculation Example

Given Data: 110010101010100100111001

Divide into 8-bit segments.

Segment 1 = 11001010
Segment 2 = 10101001
Segment 3 = 00111001

Step 1: Add First Two Segments

```
11001010
+10101001
-----
101110011
```

Since result is 9 bits, wrap around the carry.

```
01110011
   +1
-----
01110100
```

Step 2: Add Third Segment

```
01110100
+00111001
-----
10101101
```

Step 3: Take 1's Complement

10101101

1's complement:

01010010

This becomes the **Checksum**.

Step 4: Transmitted Data

Data + Checksum

Checksum sent: 01010010

Step 5: Receiver Verification

Receiver adds the given or sent data and the checksum to test if the outcome will be all 1s:

```
11001010
10101001
00111001
01010010
```

If result = 11111111

Then:

✓ No error

If result $\neq$ 11111111

Then:

✗ Error detected

Advantages of Checksum

1. Simple and easy to implement
2. Requires less computational power
3. Effective for detecting simple transmission errors

Limitations of Checksum

1. Less reliable compared to CRC
2. Cannot detect all types of errors
3. Multiple bit errors may go undetected

Error Control Methods in Data Communications

There are two common error control methods in data communication:

- (i) Forward Error Control
- (ii) Feedback Error Control (ARQ)

Forward Error Correction (FEC):

is a digital signal processing technique used in communications and storage systems to detect and correct errors in data without requiring the data to be retransmitted. It works by adding redundant data, known as error-correcting codes, to the original message before transmission over an unreliable or noisy channel.

How FEC Works

The process involves two main stages: encoding at the transmitter and decoding at the receiver.

1. **Encoding:** At the source, an FEC encoder applies a predefined mathematical algorithm to the original data stream, generating and adding redundant bits (parity bits) to form a longer codeword. This increases the total amount of data sent, an effect known as "overhead".
2. **Transmission:** The encoded data, including the redundant bits, is sent over the communication channel, where it may be subjected to noise, interference, or signal degradation, causing potential errors (bit flips or packet loss).
3. **Decoding and Correction:** At the destination, the FEC decoder uses the same algorithm and the redundant bits to check the integrity of the received data. If errors are detected within the correction limits of the specific code used, the decoder can pinpoint and fix them, recovering the original information without needing to ask the sender to resend anything.

Key Benefits

- **Improved Reliability:** Ensures data integrity in noisy environments like satellite links or long-distance fiber optic networks.
- **Reduced Latency:** Eliminates the delays associated with retransmission requests (Automatic Repeat reQuest or ARQ), which is crucial for real-time applications such as live video streaming, online gaming, and VoIP calls.
- **Bandwidth Efficiency:** Avoids the need to resend entire data packets, making more efficient use of available bandwidth, especially on lossy channels.
- **Extended Reach:** Allows communication systems to operate reliably over longer distances or with lower-cost components by compensating for signal degradation with digital correction

Feedback Error Control (ARQ)

Feedback Error Control, commonly known as **Automatic Repeat reQuest (ARQ)**, is a technique used in data transmission to ensure reliable delivery over potentially unreliable, noisy communication links. Instead of trying to correct errors on the fly, ARQ relies on the receiver identifying errors and asking the sender to retransmit the data.

How ARQ Works

1. **Error Detection:** The receiver uses error-checking codes (like Cyclic Redundancy Check - CRC) to determine if a received packet is corrupted.
2. **Feedback (ACK/NACK):**
 - **Positive Acknowledgement (ACK):** If the packet is received correctly, the receiver sends an ACK to the sender.
 - **Negative Acknowledgement (NACK):** If an error is detected, the receiver sends a NACK, signaling the sender to resend the specific data.
3. **Timeout & Retransmission:** If the sender does not receive an ACK within a set time (timeout) or receives a NACK, it assumes the data was lost or corrupted and retransmits it.

Key Types of ARQ Protocols

- **Stop-and-Wait ARQ:** The simplest form, where the sender sends one packet and waits for an ACK before sending the next. If no ACK arrives, it resends the same packet.
- **Go-Back-N ARQ:** The sender can transmit multiple packets (a window) before needing an ACK. If a frame is lost or corrupted, the receiver rejects that frame *and all subsequent frames*, causing the sender to "go back" and retransmit all of them.
- **Selective Repeat ARQ:** The most efficient form; the sender only retransmits the specific packet(s) that were corrupted or lost, rather than all subsequent packets. This requires more buffering and complex logic at the receiver.

Advantages and Disadvantages

Advantages:

- **Simplicity:** Simpler decoding mechanisms compared to forward error correction (FEC).
- **High Accuracy:** Highly reliable, as data is retransmitted until it is received correctly.
- **Bandwidth Efficiency:** In good channel conditions, it uses less bandwidth than FEC because it only sends retransmission data when errors actually occur.

Disadvantages:

- **Latency:** Retransmissions introduce delays, making it unsuitable for real-time, delay-sensitive applications like VoIP or online gaming.
- **Throughput Drop:** In very noisy channels, constant retransmissions can significantly reduce overall network throughput.